

ANÁLISE DE CÓDIGO REACT

App.jsx — Props, States, Hooks e Componentes

Disciplina: Desenvolvimento Front-End com React

Abril de 2026

1. Identificação de Props, States e Hooks

A seguir, são listados todos os momentos em que o código utiliza Props, States ou Hooks, com indicação do componente onde ocorrem e a linha de código correspondente.

1.1 States (useState)

Todos os estados estão centralizados no componente principal App e são passados como props para os filhos que precisam lê-los ou modificá-los.

State	Valor Inicial	Componente	Finalidade
nome	" (string vazia)	App	Armazena o nome digitado pelo usuário no campo de texto.
cliques	0 (número)	App	Conta quantas vezes o botão 'Incrementar' foi pressionado.
temaEscuro	false (booleano)	App	Controla se a interface está em modo escuro ou claro.

1.2 Hooks

Hook	Localização	Dependência	Ação executada
useState("")	App — linha 1 do corpo	—	Declara o state nome com setter setNome.
useState(0)	App — linha 2 do corpo	—	Declara o state cliques com setter setCliques.
useState(false)	App — linha 3 do corpo	—	Declara o state temaEscuro com setter setTemaEscuro.
useEffect(..., [cliques])	App	[cliques]	Atualiza document.title sempre que cliques muda.

1.3 Props

As props são os dados passados do componente pai (App) para cada componente filho. A tabela abaixo lista todas as props identificadas no código:

Componente Filho	Props recebidas	Tipo / Origem
Header	titulo	string — valor literal passado no JSX de App
InputUsuario	nome, setNome	string e função — state nome e seu setter
CardSaudacao	nome, temaEscuro	string e booleano — states de App
ContadorCliques	cliques, setCliques	número e função — state cliques e seu setter
ThemeToggle	dark, setDark	booleano e função — state temaEscuro e seu setter
ListaRecursos	itens	array de strings — constante recursosReact de App

2. Função Técnica de Cada Elemento

2.1 Props

Props (abreviação de properties) são o mecanismo de comunicação unidirecional do React, do componente pai para os filhos. Elas funcionam como parâmetros de função: o componente filho as recebe e as utiliza internamente, mas não pode alterá-las diretamente.

No código, as props cumprem dois papéis distintos:

- Props de leitura: transmitem um valor para que o filho exiba ou use — exemplos: título em Header, nome em CardSaudacao e itens em ListaRecursos.
- Props de ação (callbacks): transmitem a função setter do estado, permitindo que o filho solicite uma atualização do estado ao pai — exemplos: setName em InputUsuario, setCliques em ContadorCliques e setDark em ThemeToggle.

Esse padrão é chamado de "lifting state up" (elevação de estado): o estado fica no pai e os filhos o modificam indiretamente por meio das funções recebidas via prop.

2.2 States (useState)

State é a memória reativa de um componente. Quando um valor de state muda, o React re-renderiza automaticamente o componente e todos os filhos que dependem daquele dado.

O hook useState retorna um par: o valor atual do estado e uma função setter para atualizá-lo. No App.jsx, os três states gerenciam aspectos distintos da interface:

- nome: mantém o texto digitado pelo usuário em tempo real; cada tecla digitada dispara setName, que atualiza o estado e provoca re-renderização dos componentes que utilizam esse valor.
- cliques: mantém um contador inteiro; ao acionar setCliques(cliques + 1), o React calcula o novo valor, atualiza a UI e — como efeito colateral — dispara o useEffect que atualiza o título da aba.
- temaEscuro: controla um boolean que alterna duas paletas de cores; o valor é propagado como prop para CardSaudacao (que muda seu background) e usado diretamente em App para alterar o containerStyle.

2.3 useEffect

O useEffect permite executar efeitos colaterais após a renderização — operações que não fazem parte do ciclo normal de renderização do React, como manipulações do DOM, chamadas a APIs, assinaturas de eventos, etc.

A assinatura do hook aceita dois argumentos: uma função com o efeito desejado e um array de dependências. O React executa a função sempre que algum valor do array de dependências muda.

No App.jsx, o useEffect é usado da seguinte forma:

- Função executada: document.title = "Cliques: " + cliques — altera o título exibido na aba do navegador.
- Array de dependências: [cliques] — o efeito só é disparado quando o valor de cliques muda, evitando execuções desnecessárias em outras re-renderizações.

Sem esse hook, a atualização do título teria de ser feita manualmente ou ficaria fora do ciclo de vida gerenciado pelo React.

3. Análise dos 7 Componentes

O código define 6 componentes auxiliares exportados (named exports) e 1 componente principal exportado como padrão (default export). Abaixo cada um é detalhado quanto à sua responsabilidade técnica.

Componente 1 — Header

Atributo	Detalhe
Tipo de exportação	Named export (export function Header)
Props recebidas	titulo (string)
State próprio	Nenhum
Hooks	Nenhum

Função técnica: Header é um componente puramente apresentacional (stateless). Sua única responsabilidade é exibir o título da aplicação dentro de um elemento `<header>` com estilo inline fixo (fundo roxo, texto branco, centralizado). Por não possuir state nem efeitos colaterais, ele sempre renderiza o mesmo resultado para o mesmo valor de título, caracterizando uma função pura no sentido do React.

Componente 2 — InputUsuario

Atributo	Detalhe
Tipo de exportação	Named export
Props recebidas	nome (string), setNome (função)
State próprio	Nenhum (usa state do pai via props)
Hooks	Nenhum

Função técnica: InputUsuario é um componente de entrada controlada (controlled input). O valor exibido no campo de texto (`value={nome}`) é sempre idêntico ao state `nome` do App. Ao digitar, o evento `onChange` chama `setNome` com o novo valor, atualizando o state do pai — o que por sua vez re-renderiza o componente com o texto atualizado. Esse padrão garante que o React seja a única fonte de verdade para o valor do input.

Componente 3 — CardSaudacao

Atributo	Detalhe
Tipo de exportação	Named export
Props recebidas	nome (string), temaEscuro (booleano)
State próprio	Nenhum
Hooks	Nenhum

Função técnica: CardSaudacao exibe uma mensagem de boas-vindas personalizada com o nome do usuário. Caso nome seja uma string vazia (falsy), o operador || exibe o texto 'Visitante' como fallback. O componente também reage ao tema: quando temaEscuro é true, aplica fundo escuro (#333) e texto branco; caso contrário, usa fundo claro. Toda essa lógica é calculada por renderização condicional sobre o objeto estilo, sem alterar nenhum estado.

Componente 4 — ContadorClique

Atributo	Detalhe
Tipo de exportação	Named export
Props recebidas	cliques (número), setCliques (função)
State próprio	Nenhum (estado gerenciado pelo pai)
Hooks	Nenhum

Função técnica: ContadorClique exibe o valor atual do contador e oferece um botão para incrementá-lo. Ao clicar, a função anônima chama setCliques(cliques + 1), que atualiza o state cliques em App. Como consequência, App re-renderiza e passa o novo valor de cliques de volta para este componente via prop, fazendo o número exibido ser atualizado. O componente não precisa de state próprio porque o dado é compartilhado com outros componentes (o useEffect em App também depende de cliques).

Componente 5 — ThemeToggle

Atributo	Detalhe
Tipo de exportação	Named export
Props recebidas	dark (booleano), setDark (função)
State próprio	Nenhum
Hooks	Nenhum

Função técnica: ThemeToggle é um botão de alternância de tema. Ao clicar, chama setDark(!dark), invertendo o valor booleano do state temaEscuro no App. O texto do botão muda dinamicamente com uma expressão ternária: exibe 'Modo Claro' quando dark é true e 'Modo Escuro' quando dark é false, dando ao usuário uma indicação clara da ação que será executada (e não do estado atual).

Componente 6 — ListaRecursos

Atributo	Detalhe
Tipo de exportação	Named export
Props recebidas	itens (array de strings)
State próprio	Nenhum

Atributo	Detalhe
Hooks	Nenhum

Função técnica: ListaRecursos recebe um array de strings e renderiza cada elemento como um item de lista () usando o método .map(). A prop key={index} é obrigatória no React para que o algoritmo de reconciliação identifique de forma eficiente quais itens foram adicionados, removidos ou reordenados. Embora usar o índice como key seja funcional neste contexto (lista estática e sem reordenação), em listas dinâmicas o ideal seria usar um identificador único de cada item.

Componente 7 — App (componente principal)

Atributo	Detalhe
Tipo de exportação	Default export (export default function App)
Props recebidas	Nenhuma (é o componente raiz)
State próprio	nome, cliques, temaEscuro (via useState)
Hooks	useState (3x) e useEffect (1x)

Função técnica: App é o componente-raiz e o gerenciador de estado global da aplicação. Ele centraliza todos os states e os distribui para os filhos por meio de props, seguindo o padrão unidirecional de fluxo de dados do React (one-way data binding). Suas responsabilidades são:

- Declarar e gerenciar os três estados reativos da aplicação.
- Registrar o efeito colateral que sincroniza o título do navegador com o contador de cliques.
- Compor a estrutura visual da página, organizando os componentes filhos dentro da hierarquia JSX.
- Calcular o estilo do container principal de forma dinâmica, refletindo o tema selecionado pelo usuário.

Por ser o componente raiz, App não recebe props — ele é o ponto de entrada da árvore de componentes e a fonte de verdade de todos os dados compartilhados entre os componentes filhos.

Conclusão

O código App.jsx demonstra, de forma didática e funcional, os três pilares do desenvolvimento com React:

- Props: utilizadas para comunicação pai-filho, tanto para passar dados quanto para delegar a capacidade de modificar o estado (via setters).

- States: centralizados no componente App e gerenciados com `useState`, garantindo que toda mudança de dado dispare uma nova renderização reativa.
- Hooks: `useState` e `useEffect` permitem que componentes funcionais tenham memória (estado) e executem efeitos colaterais, funcionalidades que historicamente requeriam componentes de classe.

A arquitetura adotada segue boas práticas do ecossistema React: separação de responsabilidades entre componentes, fluxo de dados unidirecional e elevação de estado ao nível mínimo a mesma informação.